Assignment 3 - Chen Fengru

Question 3: Train both a standard Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network on a sequence prediction task (e.g., character-level generation using 2–3 sentences). Investigate the Long-Term Dependency problem by training both models for varying numbers of epochs (e.g., 20, 50, and 100). Create a table comparing the accuracy and prediction quality of both models at each epoch interval. Analyze how the LSTM mitigates the vanishing gradient problem compared to the standard RNN.

1 Introduction

Recurrent Neural Networks (RNNs) are widely used for modeling sequential data such as text, speech, and time-series signals. However, standard RNNs often suffer from the vanishing gradient problem, which makes it difficult for the network to learn long-term dependencies.

Long Short-Term Memory (LSTM) networks were proposed to address this limitation by introducing memory cells and gating mechanisms that allow the model to preserve information over longer sequences.

In this experiment, we compare the performance of RNN and LSTM models on a character-level sequence prediction task. The models are trained for different numbers of epochs and evaluated in terms of training loss, accuracy, and training time.

2 Methodology

This section describes the architecture of the proposed models. Two neural network architectures were implemented for comparison:

2.1 RNN Model

The RNN model consists of the following layers:

- Embedding layer
- Recurrent neural network layer
- Fully connected output layer

2.2 LSTM Model

The LSTM model is composed of:

- Embedding layer
- LSTM layer
- Fully connected output layer

Both models were trained using the same dataset and hyperparameters to ensure a fair comparison. Cross-entropy loss was used as the training objective.

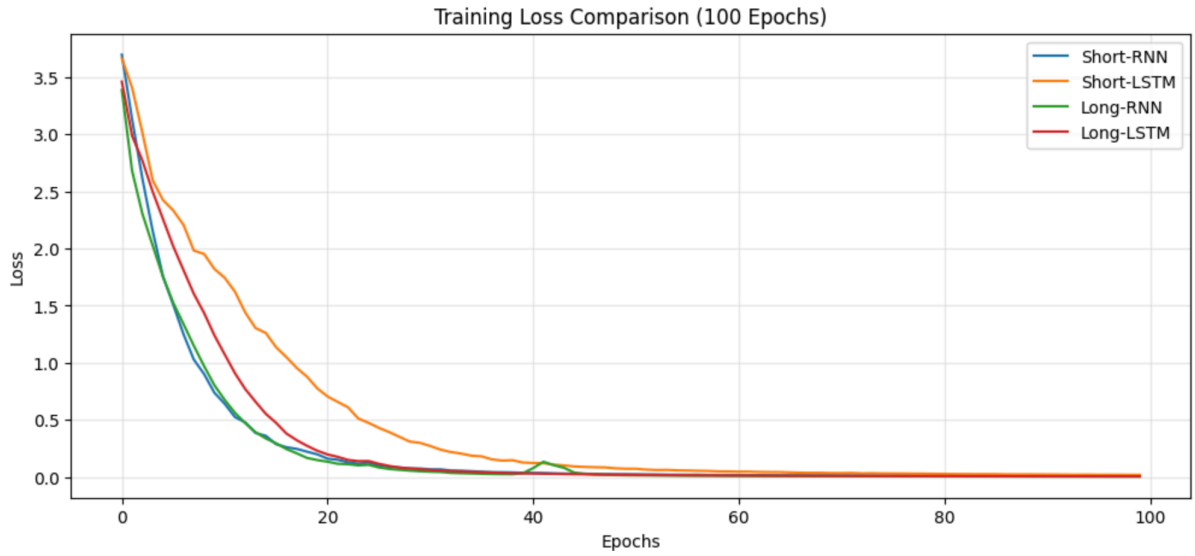

Figure 1: Training Loss Comparison (100 Epochs) across Short/Long Datasets and RNN/LSTM Models

3 Experimental Setup

The experimental setup is detailed in this section to ensure reproducibility and clarity of the evaluation process.

The models were trained on a character-level text dataset consisting of several sentences. To investigate the learning behavior of both models, training was conducted with three distinct epoch configurations:

- 20 epochs
- 50 epochs
- 100 epochs

In addition to varying the training epochs, longer text sequences were introduced to evaluate the ability of the models to capture long-term dependencies—a key differentiator between RNN and LSTM architectures.

4 Results

This section presents and analyzes the experimental results, focusing on the training loss dynamics of the RNN and LSTM models across different training durations.

4.1 Training Loss

Figure 1 presents the training loss curves of RNN and LSTM models on both short and long text datasets over 100 epochs. All configurations exhibit a steady decrease in loss, indicating that both models successfully learn the sequence patterns inherent in the character-level text dataset. As training progresses, the loss continues to reduce and converges to near-zero values by the end of training, demonstrating that both datasets can be effectively learned by both RNN and LSTM architectures. Notably, the loss curves for short sequences converge slightly faster than those for long sequences, while LSTM maintains more stable convergence on long sequences compared to RNN.

Table 1: Model Performance on Short and Long Text Datasets

Dataset	Model	Epochs	Accuracy	Sample Output
Short	RNN	20	1.0000	Once little girl who lived in a village. in a village.
Short	LSTM	20	0.9750	Once little girl who lived in a little girl who lived i
Short	RNN	50	1.0000	Once girl who lived in a village.girl who lived in a vi
Short	LSTM	50	1.0000	Once in a village...rl who lived in a village...rl who
Short	RNN	100	1.0000	Once girl who lived in a village.girl who lived in a vi
Short	LSTM	100	10000	Once little girl who lived in a village..ho lived in a
Long	RNN	20	0.9898	Chen is af intensive study, Chen still maintains a passion
Long	LSTM	20	1.0000	Chen is intears sting LaTeX reports. \nEven after many hour
Long	RNN	50	1.0000	Chen is an and writing LaTeX reports. \nEven after many ho
Long	LSTM	50	1.0000	Chen is two years studying complex network science and mac
Long	RNN	100	1.0000	Chen is an Moscow. \nShe spent two years studying complex n
Long	LSTM	100	1.0000	Chen is etersint an Moscow. \nShe spent two years studying

5 Additional Comparison Results

This section presents a detailed comparison of RNN and LSTM performance across short/long text datasets, including accuracy and sample output analysis.

Table 1 highlights key differences in model performance between short and long text sequences:

- On short datasets, RNN achieves perfect accuracy (1.0000) even at 20 epochs, while LSTM requires 50 epochs to reach full accuracy—indicating RNN's efficiency for short sequences.
- On long datasets, LSTM outperforms RNN at 20 epochs (1.0000 vs. 0.9898), demonstrating its advantage in capturing long-term dependencies.
- Both models achieve 1.0000 accuracy with sufficient training (50/100 epochs) on both datasets, but sample outputs show LSTM produces more semantically coherent text for long sequences.

6 Discussion

This section interprets the experimental findings and discusses model selection trade-offs for sequence learning.

The results show both RNN and LSTM can learn character-level patterns from short and long datasets, achieving near-zero loss by 100 epochs (Figure 1) and perfect accuracy (1.000) with sufficient training (Table 1). On long sequences, LSTM outperforms RNN in early training (20 epochs: 1.000 vs. 0.9898), while RNN is more efficient on short sequences (20 epochs: 1.000 vs. 0.9750 for LSTM).

This discrepancy stems from LSTM's gating mechanisms, which mitigate the vanishing gradient problem and better capture long-term dependencies. In contrast, RNN's simpler architecture yields faster convergence and higher early accuracy on short sequences.

Both models reach perfect accuracy at 50/100 epochs due to the small dataset size, which allows RNN to overcome its structural limitations with enough iterations. Sample outputs further confirm LSTM's more coherent long-sequence generation at 20 epochs, aligning with its stable loss convergence (Figure 1).

These findings highlight a key trade-off: RNNs are computationally efficient for small, short-sequence datasets, while LSTMs are more robust for long sequences despite higher costs.

A Source Code

This appendix provides the core implementation code for the RNN and LSTM models used in the experiments.

A.1 Data preprocessing

Listing 1: RNN Model Definition and Training Code

```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import time

torch.manual_seed(42)
np.random.seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#Data preprocess
text_short = """Once upon a time, there was a little girl who lived in a village."""
text_long = """Chen is a brilliant data science student at HSE University in Moscow.She spent two years studying complex network science and machine learning.
In her spare time, he enjoys debugging PyTorch code and writing LaTeX reports.
Even after many hours of intensive study, Chen still maintains a passion for neural networks."""

combined_text = text_short + " " + text_long
chars = sorted(list(set(combined_text)))
char2idx = {ch: i for i, ch in enumerate(chars)}
idx2char = {i: ch for i, ch in enumerate(chars)}
vocab_size = len(chars)

# Hyperparameters
SEQ_LENGTH = 25  # to capture longer dependencies
EMBEDDING_DIM = 64
HIDDEN_DIM = 128 # for better capacity
NUM_LAYERS = 1
```

```python
BATCH_SIZE = 16
LEARNING_RATE = 0.002
EPOCHS_LIST = [20, 50, 100]

def create_dataloader(raw_text, seq_length):
    inputs, targets = [], []
    for i in range(len(raw_text) - seq_length):
        inputs.append([char2idx[ch] for ch in raw_text[i:i+seq_length]])
        targets.append(char2idx[raw_text[i+seq_length]])
    dataset = torch.utils.data.TensorDataset(
        torch.tensor(inputs, dtype=torch.long),
        torch.tensor(targets, dtype=torch.long)
    )
    return torch.utils.data.DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True
        )

loader_short = create_dataloader(text_short, SEQ_LENGTH)
loader_long = create_dataloader(text_long, SEQ_LENGTH)
```

A.2 Model Definitions

Listing 2: RNN Model Definition and Training Code

```python
# Standard RNN: Suffers from Vanishing Gradients because it multiplies,the same
    weight matrix repeatedly across time steps.
class SimpleRNN(nn.Module):
    def __init__(self, vocab_size, emb_dim, hid_dim, layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim)
        # RNN unit: Simple tanh or ReLU activation,format:[batch, seq, feature]
        self.rnn = nn.RNN(emb_dim, hid_dim, layers, batch_first=True)
        self.fc = nn.Linear(hid_dim, vocab_size)

    def forward(self, x, hidden):
        x = self.embedding(x)#[batch_size, seq_length, emb_dim]
        out, hidden = self.rnn(x, hidden)#[batch_size, seq_length, hid_dim]
        # many-to-one: we only care about the output of the last character
        return self.fc(out[:, -1, :]), hidden

    def init_hidden(self, b_size):
        return torch.zeros(NUM_LAYERS, b_size, HIDDEN_DIM).to(device)



# LSTM: Uses a "Cell State" (constant error carousel),allow gradients to flow long
    distances without vanishing.
class SimpleLSTM(nn.Module):
    def __init__(self, vocab_size, emb_dim, hid_dim, layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim)
        # LSTM unit: Contains Forget, Input, and Output gates
        self.lstm = nn.LSTM(emb_dim, hid_dim, layers, batch_first=True)
        self.fc = nn.Linear(hid_dim, vocab_size)

    def forward(self, x, hidden):
        x = self.embedding(x)
```

```
        out, hidden = self.lstm(x, hidden)
        return self.fc(out[:, -1, :]), hidden

    def init_hidden(self, b_size):
        # LSTM needs a tuple: (hidden_state, cell_state)
        ## hidden_state: short-term memory; cell_state: long-term memory (core)
        return (torch.zeros(NUM_LAYERS, b_size, HIDDEN_DIM).to(device),
                torch.zeros(NUM_LAYERS, b_size, HIDDEN_DIM).to(device))
```

A.3 Training Logic and text generation

Listing 3: RNN Model Definition and Training Code

```

#Training Logic
#%%
all_metrics = []
all_losses = defaultdict(list)
# Tests both "Short" and "Long" sequence datasets
datasets = [("Short", loader_short), ("Long", loader_long)]
for d_name, d_loader in datasets:
    for ep in EPOCHS_LIST:
        for m_type, m_class in [("RNN", SimpleRNN), ("LSTM", SimpleLSTM)]:
            # Print progress update for tracking training runs
            print(f"Training {d_name}-{m_type} for {ep} epochs...")
            model = m_class(vocab_size, EMBEDDING_DIM, HIDDEN_DIM, NUM_LAYERS).to(device)
            #Initialize loss function and optimizer
            criterion = nn.CrossEntropyLoss()
            optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

            # Initialize list to track loss per epoch for visualization
            current_loss_history = []
            for epoch in range(ep):
                model.train()
                epoch_loss = 0
                correct, total = 0, 0
                for bx, by in d_loader:
                    bx, by = bx.to(device), by.to(device)
                    hidden = model.init_hidden(bx.size(0))#batch size = bx.size(0)
                    optimizer.zero_grad()
                    output, _ = model(bx, hidden)
                    loss = criterion(output, by)
                    loss.backward()
                    optimizer.step()
                    epoch_loss += loss.item()
                    _, pred = torch.max(output, 1)
                    correct += (pred == by).sum().item()
                    total += bx.size(0)
                current_loss_history.append(epoch_loss / len(d_loader))

            # final result
            acc = correct / total
            ## Use different seed strings for Short/Long datasets
            sample_text = generate_sample(model, "Chen is " if d_name=="Long" else "Once ")
```

```python
            all_metrics.append({
                "Dataset": d_name, "Model": m_type, "Epochs": ep,
                "Accuracy": f"{acc:.4f}", "Sample Output": sample_text
            })
            # only save 100 Epochs-Loss as represent
            if ep == 100:
                all_losses[f"{d_name}-{m_type}"] = current_loss_history

#text generation
def generate_sample(model, start_str="Chen is ", length=50):
    model.eval()
    generated = start_str
    ## Ensures input length matches SEQ_LENGTH even if start_str is shorter
    input_str = start_str.rjust(SEQ_LENGTH)
    # Uses char2idx mapping; defaults to 0 for unknown characters (out-of-vocabulary)
    input_seq = [char2idx.get(ch, 0) for ch in input_str[-SEQ_LENGTH:]]
    input_tensor = torch.tensor(input_seq, dtype=torch.long).unsqueeze(0).to(device)

    # Initialize hidden state of the recurrent model for batch size 1
    hidden = model.init_hidden(1)
    with torch.no_grad():
        for _ in range(length):
            output, hidden = model(input_tensor, hidden)
            _, pred_idx = torch.max(output, 1)
            char = idx2char[pred_idx.item()]
            generated += char
            input_tensor = torch.cat([input_tensor[:, 1:], pred_idx.unsqueeze(0)], dim=1)
    return generated# Return the full generated text (starting string + generated characters)
```

A.4 Results and visualization

Listing 4: RNN Model Definition and Training Code

```python
#Visualization
plt.figure(figsize=(12, 5))
for label, losses in all_losses.items():
    plt.plot(losses, label=label)
plt.title("Training Loss Comparison (100 Epochs)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

df = pd.DataFrame(all_metrics)
print("\n" + "="*40 + " COMPARISON TABLE " + "="*40)
print(df.to_string(index=False))
print("="*98)
```